HashiCorp Vault — Day 2: Dynamic Secrets, Auth & Policies

Study material · Oct 16, 2026 · @Vivek Arora

Day 2 at a glance

Day 2 moves from static secrets to credentials Vault creates on demand and destroys on schedule, and to the machine identities and policies that decide who may ask for them.

Learning objectives

1. Explain why dynamic secrets beat static ones, and configure the Database, AWS and Azure secrets engines.
2. Write ACL policies with path globs, segment wildcards, templating, parameter constraints and `deny`, following least privilege.
3. Configure and use AppRole, Kubernetes and JWT/OIDC auth methods for machines, pods, pipelines and humans.
4. Inspect, renew and revoke leases; explain TTL, max TTL and revoke-prefix.
5. Deliver secrets safely with response wrapping and the cubbyhole engine.
6. Build an end-to-end flow: an AppRole application that receives a wrapped `secret_id` and reads short-lived PostgreSQL credentials.

Suggested agenda

#	Module	Duration	Labs
1	Dynamic secrets: Database, AWS, Azure	90 min	Labs 2.1 and 2.2 (optional)
2	Policies and access control	60 min	Lab 2.3
3	Auth deep dive: AppRole, Kubernetes, JWT/OIDC	90 min	Labs 2.4, 2.5 and 2.6 (optional)
4	Leases, renewal and revocation	45 min	Lab 2.7
5	Response wrapping and cubbyhole	45 min	Lab 2.8
6	Capstone lab	60 min	Lab 2.9

Lab prerequisites

- The Day 1 environment, or a dev server: `vault server -dev -dev-root-token-id=root`.
- Docker (for PostgreSQL and, optionally, `kind` for Kubernetes).
- `psql` client (`sudo apt install -y postgresql-client`), `jq`, `curl`, Python 3 with `pip install hvac psycopg2-binary`.
- Optional cloud accounts: an AWS sandbox account with an IAM user for Vault; an Azure subscription with a service principal.
- `kubectl`, `kind` and `helm` for the Kubernetes auth lab.

```
# Quick start for every lab in this guide
$ vault server -dev -dev-root-token-id=root &          # or use your Day 1 Raft server
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root
```

Hands-on: every lab step, with expected-output screenshots, lives in the Vault Lab Guide — Day 1 & Day 2. Each module below ends with the lab that practises it. Labs 2.1, 2.3, 2.4 and 2.7 build on one another, so run them in a single dev-server session.

All passwords, tokens and keys shown are lab-only examples — never reuse them.

Module 1 — Dynamic secrets engines: Database, AWS, Azure

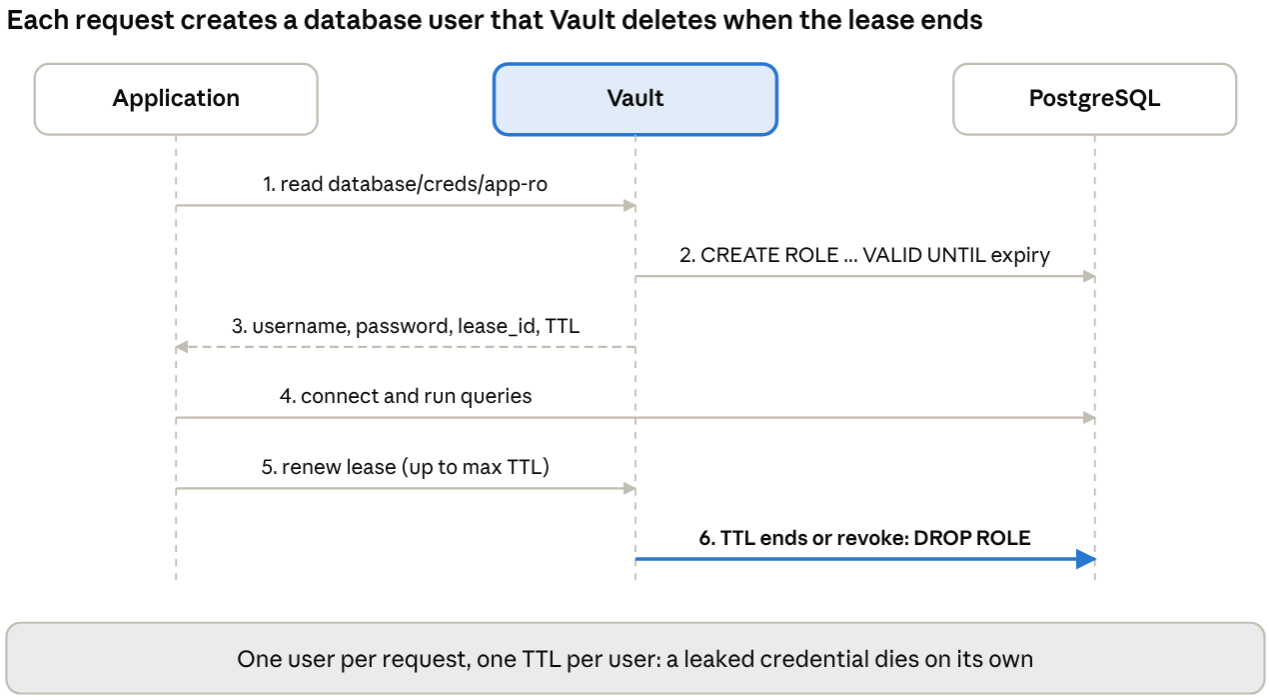
A dynamic secret is created at the moment it is requested, is unique to the requester, and is destroyed automatically when its lease ends — so there is nothing long-lived to leak, share or forget to rotate.

Static vs dynamic secrets

	Static secret (KV)	Dynamic secret
Created by	A human, once	Vault, on every request
Shared between apps	Often	Never — one credential per requester
Lifetime	Until someone rotates it	Lease TTL (minutes to hours)
Revocation	Manual, affects every user of it	Per lease, or per prefix in one command
Audit trail	Who read the shared value	Exactly which identity got which credential

	Static secret (KV)	Dynamic secret
Blast radius of a leak	Everything that uses it, until rotated	One user, until its TTL ends

How a dynamic secret works



Dynamic database credential lifecycle · 6 messages

Vault holds one privileged connection to the target system and uses it only to create and delete short-lived identities; applications never see the privileged credential.

Database secrets engine

Object	Path	What it holds
Connection	`database/config/<name>`	Plugin, connection URL, Vault's admin user, `allowed_roles`
Dynamic role	`database/roles/<role>`	SQL to create and revoke users, `default_ttl`, `max_ttl`
Credentials	`database/creds/<role>`	Read it to get a new username/password + lease
Static role	`database/static-roles/<role>`	An existing DB user whose password Vault rotates on a schedule

Object	Path	What it holds
Root rotation	`database/rotate-root/<name>`	Changes Vault's admin password so only Vault knows it

Supported plugins include PostgreSQL, MySQL/MariaDB, Microsoft SQL Server, Oracle, MongoDB, Redis, Snowflake, Cassandra and Elasticsearch.

> **Do Lab 2.1 — PostgreSQL dynamic secrets** (35 min). PostgreSQL in Docker, dynamic read-only and read-write users, lease revocation, a static role for a legacy account, and root rotation.

Key commands for reference (Vault connects as a dedicated `vaultsvc` account so that `rotate-root` never locks out your own admin login):

```
$ vault secrets enable database
$ vault write database/config/appdb \
    plugin_name=postgresql-database-plugin \
    connection_url="postgresql://{{username}}:
{{password}}@127.0.0.1:5432/appdb?sslmode=disable" \
    username="vaultsvc" password="VaultSvc1n" \
    allowed_roles="app-ro,app-rw,payments-ro,legacy-report"
$ vault write database/roles/app-ro db_name=appdb \
    creation_statements=@app-ro.sql revocation_statements=@revoke.sql \
    default_ttl=1h max_ttl=24h
$ vault read database/creds/app-ro                      # new user + lease
on every read
$ vault write database/static-roles/legacy-report db_name=appdb
username=report_user rotation_period=24h
$ vault write -f database/rotate-root/appdb             # only Vault knows
vaultsvc's password now
```

Revocation SQL must `REVOKE` grants before `DROP ROLE`, or PostgreSQL refuses to drop the user; the exact SQL files are in Lab 2.1, Step 3.

Static roles rotate the password of a user that already exists (service accounts, legacy apps that cannot change usernames) — see Lab 2.1, Steps 8–9.

MySQL equivalent of the read-only role:

```
$ vault write database/config/shopdb plugin_name=mysql-database-plugin \
    connection_url="{{username}}:{{password}}@tcp(127.0.0.1:3306)/" \
    username="vaultadmin" password="VaultAdm1n" allowed_roles="shop-ro"
$ vault write database/roles/shop-ro db_name=shopdb default_ttl=1h
max_ttl=24h \
    creation_statements="CREATE USER '{{name}}'@'%' IDENTIFIED BY
'{{password}}'; GRANT SELECT ON shop.* TO '{{name}}'@'%';"
```

From Python (`pip install hvac psycopg2-binary`):

```
# db_dynamic.py — get a fresh PostgreSQL user, use it, then hand it back
import os
import hvac
import psycopg2

client = hvac.Client(url=os.environ["VAULT_ADDR"],
token=os.environ["VAULT_TOKEN"])

creds = client.secrets.database.generate_credentials(name="app-ro")
user, password = creds["data"]["username"], creds["data"]["password"]
lease_id, ttl = creds["lease_id"], creds["lease_duration"]
print(f"Got {user} for {ttl}s (lease {lease_id})")

with psycopg2.connect(host="127.0.0.1", dbname="appdb", user=user,
password=password) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT id, item FROM orders")
        print(cur.fetchall())

client.sys.renew_lease(lease_id=lease_id, increment=3600)   # extend if still
needed
client.sys.revoke_lease(lease_id=lease_id)                   # or revoke when
done
```

AWS secrets engine

Credential type	What Vault returns	Typical use
`iam_user`	A new IAM user + access key, deleted at lease end	Tools that need long-ish keys
`assumed_role`	STS credentials from `sts:AssumeRole`	Preferred: no IAM users created, cross-account roles

Credential type	What Vault returns	Typical use
`federation_token`	STS federation token	Scoped-down temporary access
`session_token`	STS session token (optionally with MFA)	Short sessions for the Vault IAM user itself

```
$ vault secrets enable aws
$ vault write aws/config/root \
    access_key=AKIAEXAMPLE secret_key=EXAMPLEKEY region=ap-south-1
$ vault write -f aws/config/rotate-root          # optional: only Vault knows
the key afterwards

# iam_user role with an AWS-managed policy
$ vault write aws/roles/s3-readonly credential_type=iam_user \
    policy_arns=arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
$ vault read aws/creds/s3-readonly

# assumed_role role (recommended)
$ vault write aws/roles/deploy credential_type=assumed_role \
    role_arns=arn:aws:iam::123456789012:role/vault-deploy \
    default_sts_ttl=1h max_sts_ttl=4h
$ vault write aws/sts/deploy ttl=30m

# Use them
$ export AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=...
AWS_SESSION_TOKEN=...
$ aws sts get-caller-identity
```

New IAM access keys can take a few seconds to become usable (IAM is eventually consistent); retry the first call. Minimum IAM permissions for the Vault user when using `iam_user`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iam:CreateUser", "iam:DeleteUser",
        "iam:CreateAccessKey", "iam:DeleteAccessKey", "iam:ListAccessKeys",
        "iam:AttachUserPolicy", "iam:DetachUserPolicy",
"iam:ListAttachedUserPolicies",
        "iam:PutUserPolicy", "iam:DeleteUserPolicy", "iam:ListUserPolicies",
        "iam:ListGroupsForUser", "iam:RemoveUserFromGroup"
```

```
      ],
      "Resource": "arn:aws:iam::123456789012:user/vault-*"
    },
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::123456789012:role/vault-*"
    }
  ]
}
```

Azure secrets engine

Vault creates a service principal (or a new secret on an existing application), assigns it Azure RBAC roles, and deletes it when the lease ends.

```
$ vault secrets enable azure
$ vault write azure/config \
    subscription_id=<subscription-id> tenant_id=<tenant-id> \
    client_id=<vault-sp-app-id> client_secret=<vault-sp-secret>

# Role: a Contributor service principal scoped to one resource group
$ vault write azure/roles/rg-contributor ttl=1h max_ttl=4h azure_roles=-<<EOF
[
  {
    "role_name": "Contributor",
    "scope": "/subscriptions/<subscription-id>/resourceGroups/rg-training"
  }
]
EOF

$ vault read azure/creds/rg-contributor            # returns client_id +
client_secret
$ az login --service-principal -u <client_id> -p <client_secret> --tenant
<tenant-id>
$ az group show -n rg-training

# Existing-application mode: Vault only rotates secrets on an app you created
$ vault write azure/roles/existing-app application_object_id=<app-object-id>
ttl=1h
```

The service principal Vault itself uses needs Microsoft Graph permission to manage applications (for example `Application.ReadWrite.OwnedBy`) and a role that can assign RBAC (Owner or User Access Administrator) on the target scope. Role assignments can take a minute to propagate.

> **Do Lab 2.2 — AWS and Azure dynamic secrets** (optional, 20 min, needs a sandbox account). Issue IAM-user and STS credentials, and a short-lived Azure service principal, then revoke them.

Choosing an engine

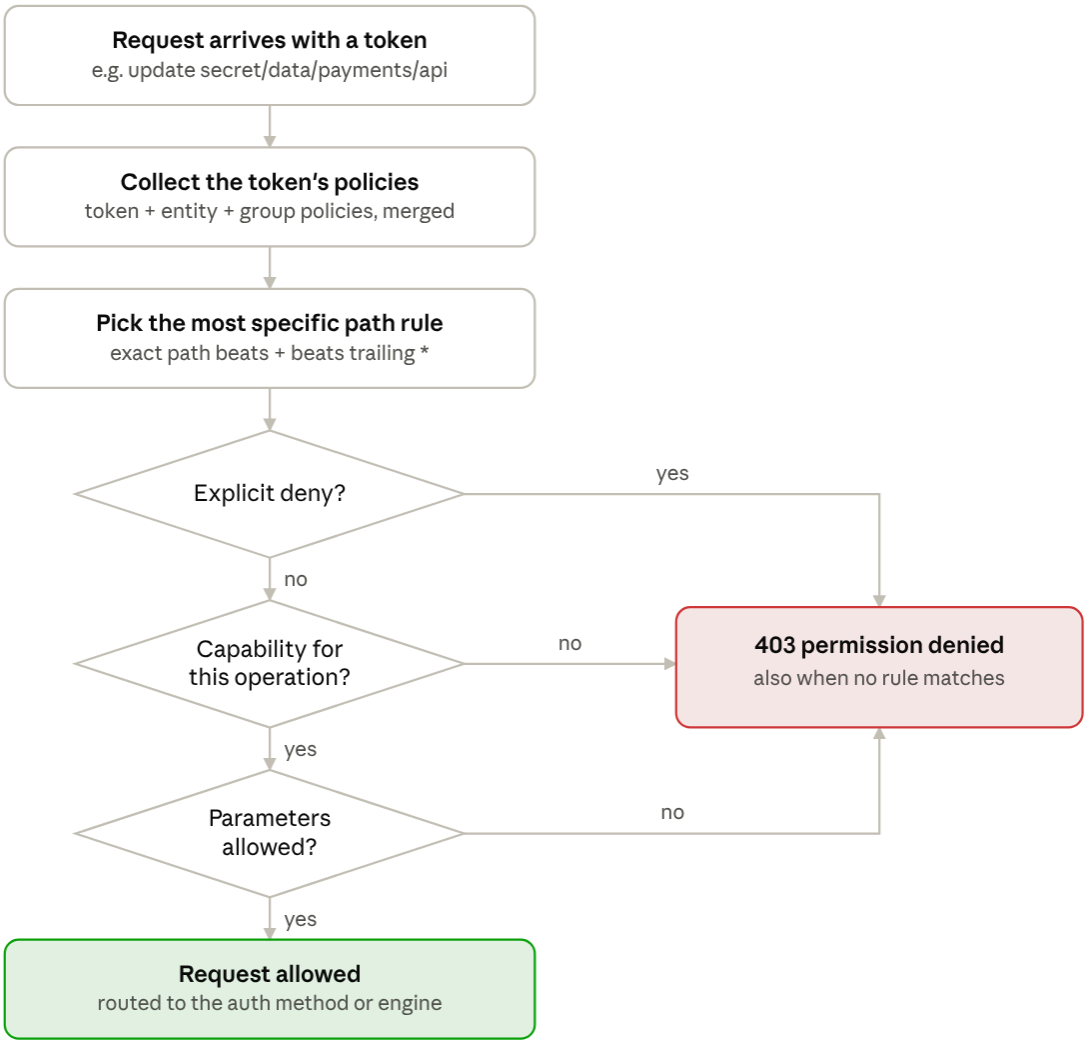
	Database	AWS	Azure
Creates	DB users	IAM users or STS credentials	Service principals or app secrets
Vault's own credential	DB admin user	IAM user or instance role	Service principal
Fastest option	Dynamic role	`assumed_role` (no IAM objects)	Existing-application mode
Watch out for	Grants on tables created later	IAM eventual consistency	RBAC propagation delay

Module 2 — Policies and access control

Policies are deny-by-default rules of *path + capabilities*; a token can do exactly what its merged policies grant on the most specific matching path, and an explicit `deny` always wins.

How Vault evaluates a request

Anything not explicitly allowed is denied



The root token skips this check entirely, which is why it should be revoked after setup.

Policy evaluation · 3 checks, deny by default

Most 403s come from the third box: a more specific rule matched than the one you expected, and it lacks the capability you thought you granted.

Anatomy of a policy

```
# payments-app.hcl
path "secret/data/payments/*" {
  capabilities = ["read"]
}

path "secret/metadata/payments/*" {
  capabilities = ["list"]
}

path "database/creds/payments-ro" {
  capabilities = ["read"]
}

# Never allow this app to touch the payments signing key
path "secret/data/payments/signing-key" {
  capabilities = ["deny"]
}
```

Capability	HTTP verb	Meaning
`create`	POST / PUT	Write data where none exists
`read`	GET	Read data
`update`	POST / PUT	Change existing data (most config endpoints)
`patch`	PATCH	Partial update (e.g. `vault kv patch`)
`delete`	DELETE	Delete data
`list`	LIST	List keys under a path
`sudo`	—	Required in addition for root-protected paths such as `sys/audit` and `sys/raw`
`deny`	—	Overrides every other capability on that path

Path matching rules

Pattern	Matches	Does not match
`secret/data/app`	Exactly `secret/data/app`	`secret/data/app/db`
`secret/data/app/*`	Anything under `app/`, any depth	`secret/data/app` itself

Pattern	Matches	Does not match
`secret/data/app*`	`app`, `app1`, `app/db`, `apple` (prefix match)	`ap`
`secret/data/+/config`	`secret/data/team1/config`	`secret/data/team1/x/config`

When several rules match a request, Vault uses only the **most specific** one: the rule whose first wildcard appears later wins; a rule without a trailing `*` beats one with it; fewer `+` segments win; then the longer path wins. Rules for the identical path string across different policies are merged first, and `deny` in any of them wins.

```
# Gotcha: the specific rule wins, so this token CANNOT update config
path "secret/data/app/*" {
  capabilities = ["create", "read", "update"]
}
path "secret/data/app/config" {
  capabilities = ["read"]          # more specific -> only read here
}
```

Templated policies (one policy, many users)

```
# per-user.hcl — every user gets a private folder named after their login
path
"secret/data/users/{{identity.entity.aliases.auth_userpass_6a1b2c3d.name}}/*"
{
  capabilities = ["create", "read", "update", "patch", "delete"]
}
path
"secret/metadata/users/{{identity.entity.aliases.auth_userpass_6a1b2c3d.name}
}/*" {
  capabilities = ["list", "read", "delete"]
}
```

```
# Find the mount accessor to put in the template
$ vault auth list -format=json | jq -r '.["userpass/"].accessor'
auth_userpass_6a1b2c3d
```

Other useful parameters: `{{identity.entity.id}}`, `{{identity.entity.name}}`, `{{identity.entity.metadata.<key>}}` and `{{identity.groups.names.<group>.id}}`.

Fine-grained controls

```
# helpdesk.hcl — may create users and reset passwords, but only hand out dev-read
path "auth/userpass/users/*" {
  capabilities       = ["create", "update"]
  allowed_parameters = {
    "password"       = []
    "token_policies" = ["dev-read"]
  }
}

# Wrapping can be forced: secret_ids must be fetched wrapped for 1-10 minutes
path "auth/approle/role/payments/secret-id" {
  capabilities     = ["update"]
  min_wrapping_ttl = "1m"
  max_wrapping_ttl = "10m"
}
```

Control	Effect
`allowed_parameters`	Only these keys (and, if listed, only these values) may be sent; `"*" = []` allows any key
`denied_parameters`	These keys or values are rejected
`required_parameters`	The request must include these keys
`min_wrapping_ttl` / `max_wrapping_ttl`	The response must be wrapped, within these limits

Least-privilege patterns

Persona	Grant	Never grant
Application	`read` on its own KV path and DB role	`list` on whole mounts, `sys/*`
CI/CD deployer	`read` on deploy secrets; `update` on `auth/approle/role/<app>/secret-id`	`sudo`, policy writes
Platform/ops admin	Mount, auth and policy management on named paths	Reading application secret values
Helpdesk	User creation with `allowed_parameters`	Arbitrary `token_policies`

Persona	Grant	Never grant
Break-glass	Broad access, kept on an offline userpass/token with alerts	Daily use

Two rules of thumb: prefer many small policies attached to identity groups over one large policy, and never edit the built-in `default` policy casually — every token gets it unless created with `-no-default-policy`.

Hands-on

> **Do Lab 2.3 — Least-privilege policies and identity groups** (25 min). Test `payments-app` with `vault token capabilities`, see the specificity gotcha, inspect the resultant ACL, and grant policies through an identity group and a templated per-user policy.

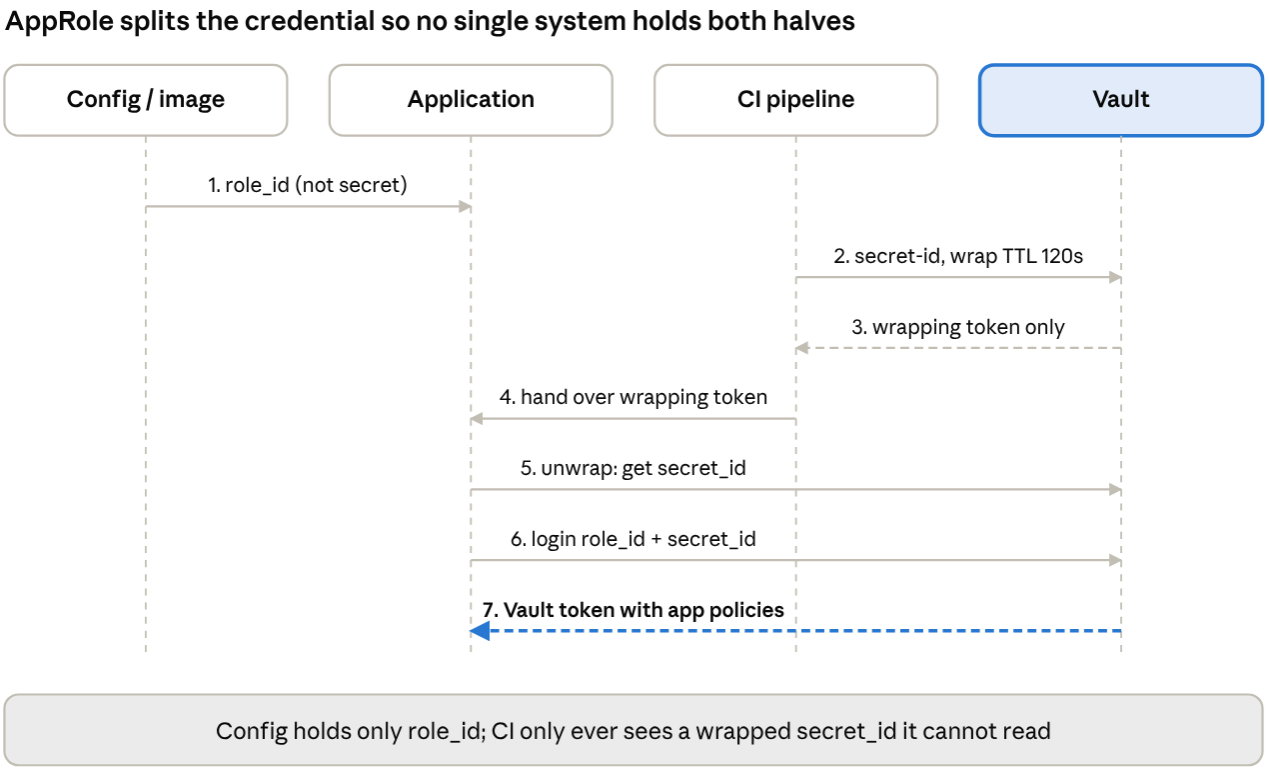
Module 3 — Auth methods deep dive: AppRole, Kubernetes, JWT/OIDC

Machines should never log in with a password a human typed; each method here lets a workload prove who it is with something its platform already guarantees — a split credential, a service account, or a signed token from an identity provider.

Method	Who logs in	What proves identity	Vault trusts
AppRole	VMs, legacy apps, scripts	`role_id` + `secret_id`	Whoever delivered the `secret_id` (your CI/orchestrator)
Kubernetes	Pods	Service account JWT	The Kubernetes API (TokenReview)
JWT	CI jobs (GitHub Actions, GitLab), workloads with OIDC tokens	A signed JWT and its claims	The issuer's signing keys (JWKS / discovery)
OIDC	Humans in a browser	Login at Entra ID, Okta, Google	The identity provider

AppRole

`role_id` works like a username and is not secret; `secret_id` works like a password and should be short-lived, single-use and delivered separately.



AppRole secure introduction · 7 messages

This pattern is called *secure introduction*: the CI pipeline is trusted to request a `secret_id`, but because it is wrapped, the pipeline never sees the value and any interception is detectable.

Role parameter	Purpose	Example
`token_policies`	Policies on the resulting token	`payments-app`
`token_ttl` / `token_max_ttl`	Token lifetime and renewal cap	`1h` / `4h`
`secret_id_ttl`	How long an unused `secret_id` stays valid	`10m`
`secret_id_num_uses`	Logins allowed per `secret_id`	`1`
`secret_id_bound_cidrs` / `token_bound_cidrs`	Only usable from these networks	`10.0.0.0/16`

Role parameter	Purpose	Example
`bind_secret_id`	Require a `secret_id` at all (keep `true`)	`true`

```
$ vault auth enable approle
$ vault write auth/approle/role/payments \
    token_policies=payments-app token_ttl=1h token_max_ttl=4h \
    secret_id_ttl=10m secret_id_num_uses=1 \
    secret_id_bound_cidrs="127.0.0.1/32,10.0.0.0/16"

# role_id: fetch once, put in app config
$ vault read -field=role_id auth/approle/role/payments/role-id

# secret_id: plain (lab) or wrapped (real life)
$ vault write -f auth/approle/role/payments/secret-id
$ vault write -wrap-ttl=120s -f auth/approle/role/payments/secret-id

# Log in
$ vault write auth/approle/login role_id=<role_id> secret_id=<secret_id>

# Manage secret_ids by accessor (never list the secrets themselves)
$ vault list auth/approle/role/payments/secret-id
$ vault write auth/approle/role/payments/secret-id-accessor/destroy
secret_id_accessor=<accessor>
```

Policy for the CI pipeline — it may only mint wrapped `secret_id`s for this one role:

```
# ci-payments.hcl
path "auth/approle/role/payments/secret-id" {
  capabilities     = ["update"]
  min_wrapping_ttl = "1s"
  max_wrapping_ttl = "300s"
}
```

Vault Agent removes token handling from the app: it logs in with AppRole, keeps the token renewed, and renders secrets into a file. Day 3 Module 4 covers the rest of its feature set: PKI certificates, run-a-command on change, process supervisor mode and the API proxy.

```
# agent.hcl
vault {
  address = "http://127.0.0.1:8200"
}
```

```
auto_auth {
  method "approle" {
    config = {
      role_id_file_path                   = "/etc/vault/role_id"
      secret_id_file_path                 = "/etc/vault/secret_id"
      remove_secret_id_file_after_reading = true
    }
  }
  sink "file" {
    config = { path = "/etc/vault/token" }
  }
}

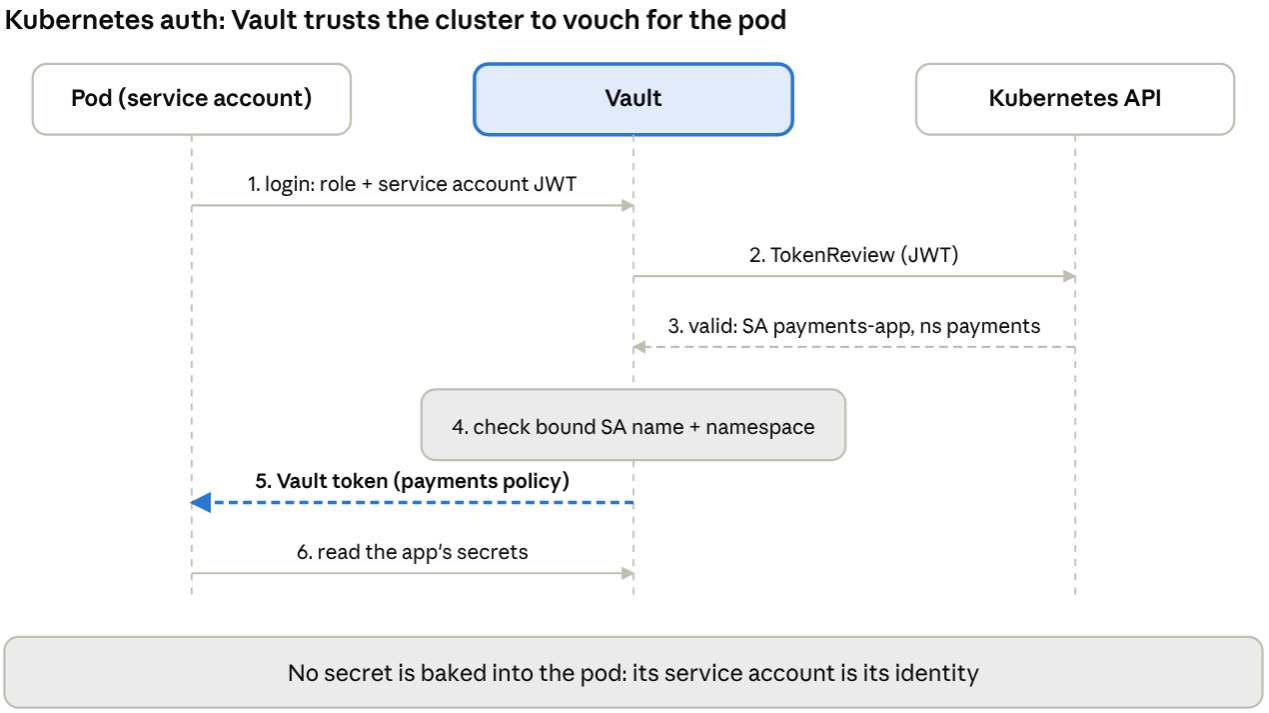
template {
  source      = "/etc/vault/db.ctmpl"
  destination = "/etc/app/db.env"
}
```

```
{{/* /etc/vault/db.ctmpl — re-rendered before the dynamic credential expires
*/}}
{{ with secret "database/creds/payments-ro" }}
DB_USER={{ .Data.username }}
DB_PASS={{ .Data.password }}
{{ end }}
```

```
$ vault agent -config=agent.hcl
```

> **Do Lab 2.4 — AppRole and Vault Agent** (30 min). Single-use `secret_id`s, a CI policy that forces wrapping, and Vault Agent rendering dynamic database credentials into a file.

Kubernetes



Kubernetes auth login · 6 steps

Vault calls the cluster's TokenReview API to confirm the service account token is real, then checks it against the role's bound service account names and namespaces before issuing a token.

> **Do Lab 2.5 — Kubernetes auth with kind and the Agent Injector** (35 min). A kind cluster, Vault installed with Helm, a Kubernetes auth role bound to one service account, and a pod that receives `API_KEY` through the injector sidecar.

Vault runs inside the cluster here, so it reviews tokens with its own service account. A Vault outside the cluster also needs `token_reviewer_jwt` and `kubernetes_ca_cert` in `auth/kubernetes/config`, from a service account bound to the `system:auth-delegator` ClusterRole.

Alternatives to the injector: the **Vault Secrets Operator** syncs Vault secrets into native Kubernetes Secrets, and the **CSI provider** mounts them as volumes. Day 3 Module 9 compares all three and runs Vault itself on Kubernetes as an HA Raft cluster (Lab 3.9).

JWT (machines) and OIDC (humans)

Both use the same plugin. JWT verifies a token the client already has; OIDC runs a browser redirect to the identity provider.

GitHub Actions without stored secrets:

```
$ vault auth enable jwt
$ vault write auth/jwt/config \
    oidc_discovery_url="https://token.actions.githubusercontent.com" \
    bound_issuer="https://token.actions.githubusercontent.com"

$ vault write auth/jwt/role/gha-deploy - <<EOF
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["vault"],
  "bound_claims": {
    "repository": "stackmaster/payments-api",
    "ref": "refs/heads/main"
  },
  "token_policies": ["payments-deployer"],
  "token_ttl": "10m"
}
EOF
```

```
# .github/workflows/deploy.yml
permissions:
  id-token: write        # lets the job request an OIDC token
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: hashicorp/vault-action@v3
        with:
          url: https://vault.example.com:8200
          method: jwt
          path: jwt
          role: gha-deploy
          jwtGithubAudience: vault
          secrets: |
            secret/data/payments/api key | API_KEY
      - run: ./deploy.sh   # API_KEY is available as an env var
```

`bound_claims` is what makes this safe: only the `main` branch of that one repository can obtain the token. GitLab CI works the same way with `oidc_discovery_url="https://gitlab.com"` and claims such as `project_path` and `ref_protected`.

Human SSO with Microsoft Entra ID:

```
$ vault auth enable oidc
$ vault write auth/oidc/config \
    oidc_discovery_url="https://login.microsoftonline.com/<tenant-id>/v2.0" \
    oidc_client_id="<app-client-id>" oidc_client_secret="<client-secret>" \
    default_role=engineer

$ vault write auth/oidc/role/engineer \
    user_claim=preferred_username groups_claim=groups \
    oidc_scopes="openid,profile,email" token_policies=default token_ttl=8h \
    allowed_redirect_uris="http://localhost:8250/oidc/callback" \

allowed_redirect_uris="https://vault.example.com:8200/ui/vault/auth/oidc/oidc
/callback"

$ vault login -method=oidc role=engineer        # opens a browser

# Map an Entra ID group to Vault policies
$ GID=$(vault write -field=id identity/group name=platform-engineers
type=external policies=platform-admin)
$ ACC=$(vault auth list -format=json | jq -r '.["oidc/"].accessor')
$ vault write identity/group-alias name=<entra-group-object-id>
mount_accessor=$ACC canonical_id=$GID
```

> **Do Lab 2.6 — JWT for GitHub Actions and OIDC login** (optional, 25 min). A workflow that reads a secret with no stored credential, a branch that is refused, and an Entra ID login whose policies come from a group.

Which method when

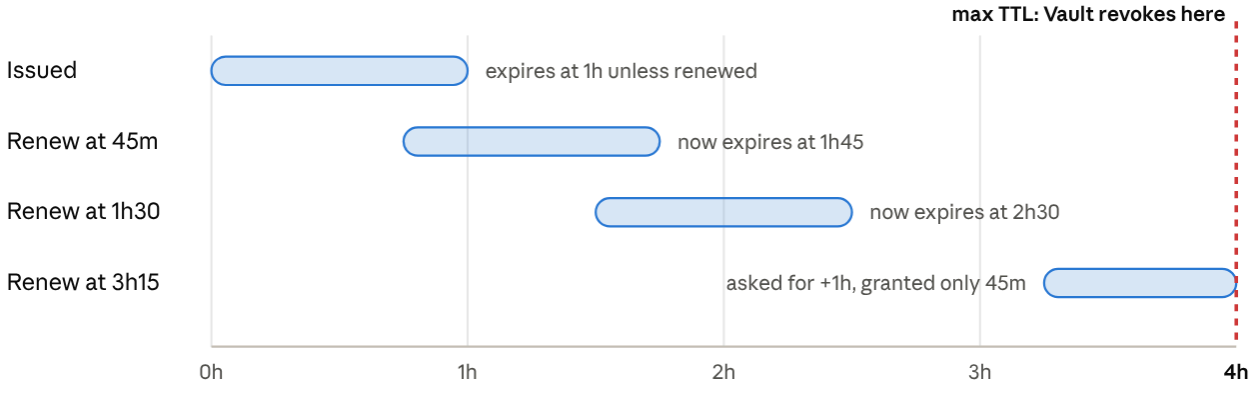
Workload	Use	Why
Pod on Kubernetes	Kubernetes auth (+ Agent Injector or VSO)	Nothing to distribute; identity is the service account
GitHub Actions / GitLab CI	JWT	No stored Vault credentials in the CI system
VM or bare-metal app	AppRole with wrapped `secret_id`, or cloud auth (AWS/Azure)	Works anywhere; cloud auth needs no secret at all
Engineers	OIDC	SSO, MFA and offboarding handled by the IdP
Break-glass	Userpass or token kept offline	Works when the IdP is down

Module 4 — Leases, renewal and revocation

Every dynamic secret and every service token carries a lease: a promise that Vault will revoke it at a known time, which also lets you revoke one credential, a whole role, or a whole engine in a single command.

Renewals slide the expiry forward, but never past max TTL

Lease with TTL 1h and max TTL 4h, renewed by the application



After max TTL the app must request a new credential (a new lease); Vault Agent does this for you.

Lease timeline · TTL 1h, max TTL 4h

Renewal is relative to *now*, not to the old expiry, and the max TTL is a hard ceiling counted from the moment the lease was created.

Lease vocabulary

Field	Meaning
`lease_id`	Unique ID such as `database/creds/app-ro/Xy7Q...`; its prefix is the path that created it
`lease_duration`	Seconds until expiry (the current TTL)
`lease_renewable`	Whether `renew` is allowed
increment	TTL requested on renewal; Vault may grant less
max TTL	Absolute cap from creation; renewals cannot pass it

Where TTLs come from

The effective TTL is the smallest limit in this chain:

Level	Setting	Default
System	`default_lease_ttl` / `max_lease_ttl` in `vault.hcl`	768h (32 days)
Mount	`vault secrets tune -default-lease-ttl=... -max-lease-ttl=... <mount>/`	Inherits system
Role	`default_ttl` / `max_ttl` on the role, `token_ttl` / `token_max_ttl` on auth roles	Inherits mount
Request	`-increment` on renew, `ttl=` on some engines	Role default

```
$ vault secrets tune -default-lease-ttl=30m -max-lease-ttl=2h database/
$ vault read sys/mounts/database/tune
```

Hands-on

> **Do Lab 2.7 — Leases, renewal and revocation** (20 min). Inspect and renew leases, hit the max-TTL cap, revoke by prefix, and watch a database user vanish when the token that created it is revoked.

Leases belong to the token that created them. Revoke the token and Vault revokes its child tokens and every lease they obtained (Lab 2.7, Step 6).

That is why an application's DB credentials disappear when its Vault token expires — and why long-running apps must renew their token as well as their leases.

Revocation playbook

Incident	Command
One credential leaked	`vault lease revoke <lease_id>`
A role's credentials compromised	`vault lease revoke -prefix database/creds/<role>`
Everything from one engine	`vault lease revoke -prefix database/` then `vault write -f database/rotate-root/<conn>`
A token leaked	`vault token revoke <token>` or `vault token revoke -accessor <accessor>`
Every token from one auth mount	`vault lease revoke -prefix auth/approle/`
Backend unreachable, lease stuck	`vault lease revoke -force -prefix <path>` — removes it from Vault only; clean the backend by hand

Renewing from code

```
# lease_keeper.py — renew at 2/3 of the TTL, fetch new creds when the cap is
reached
import os
import time
import hvac

client = hvac.Client(url=os.environ["VAULT_ADDR"],
token=os.environ["VAULT_TOKEN"])

def new_creds():
    c = client.secrets.database.generate_credentials(name="app-ro")
    print("new user:", c["data"]["username"])
    return c["lease_id"], c["lease_duration"]

lease_id, ttl = new_creds()
while True:
    time.sleep(max(ttl * 2 // 3, 5))
    renewed = client.sys.renew_lease(lease_id=lease_id, increment=3600)
    granted = renewed["lease_duration"]
    print(f"renewed {lease_id}: {granted}s left")
```

```
    if granted < 3600:                # max TTL is near: rotate before it
expires
        client.sys.revoke_lease(lease_id=lease_id)
        lease_id, ttl = new_creds()    # reconnect the DB pool with the new
user here
    else:
        ttl = granted
```

In production, let Vault Agent (Module 3) or the Kubernetes injector do this: they renew tokens and leases and re-render templates before expiry.

Module 5 — Response wrapping and cubbyhole

Response wrapping replaces a secret in transit with a single-use, short-lived token that points to it; the cubbyhole engine is the per-token private storage that makes this possible.

Cubbyhole: private storage per token

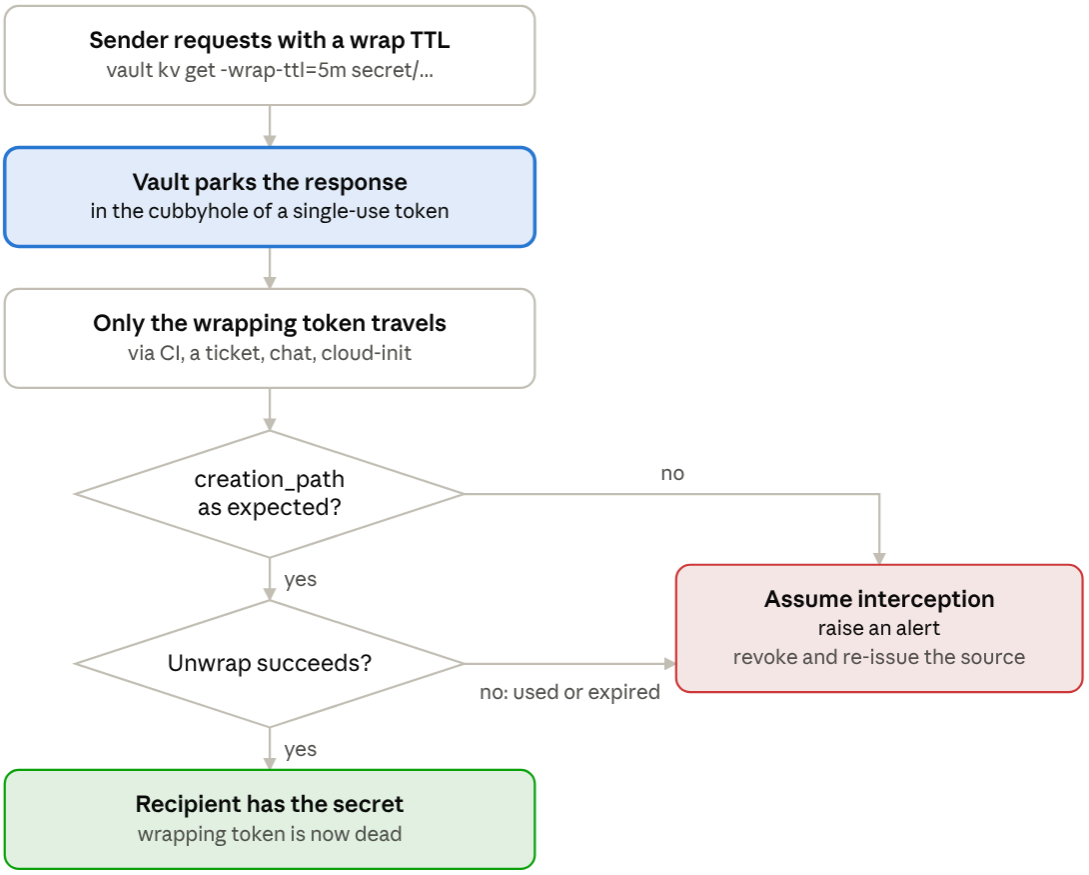
Property	Detail
Scope	Each token has its own `cubbyhole/`; no other token can read it — not even root
Lifetime	Destroyed when the token expires or is revoked
Mount	Always at `cubbyhole/`; cannot be disabled, moved or tuned
Access	Granted by the built-in `default` policy

```
$ vault write cubbyhole/notes msg="only my token can see this"
$ vault read cubbyhole/notes

$ T2=$(vault token create -policy=default -ttl=10m -field=token)
$ VAULT_TOKEN=$T2 vault read cubbyhole/notes     # No value found: different
token, different cubbyhole
$ VAULT_TOKEN=$T2 vault write cubbyhole/notes msg="T2's own note"
$ vault token revoke $T2                         # T2's cubbyhole is gone
with it
```

Response wrapping

A wrapped secret can be opened once, so interception is always detected



The secret itself never appears in logs, pipelines or chat; only a pointer that works once and expires.

Response wrapping · 2 integrity checks

Wrapping gives you three guarantees: the secret is not exposed in transit (only a reference is), it can be read exactly once, and if someone else read it first the legitimate recipient's unwrap fails — which is your alarm.

Field in `wrap_info`	Use
`token`	The single-use wrapping token to hand over
`accessor`	Lets an operator revoke the wrapping token without seeing it
`ttl`	Seconds before it self-destructs
`creation_time`	When it was minted
`creation_path`	The path that produced the wrapped response — verify it before unwrapping

> **Do Lab 2.8 — Response wrapping and cubbyhole** (20 min). Prove cubbyhole isolation, wrap and verify a secret, unwrap it once, see the second unwrap fail, and simulate an interception in pairs.

Through the API, ask for wrapping with a header and unwrap with the wrapping token itself:

```
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" -H "X-Vault-Wrap-TTL: 300" \
    $VAULT_ADDR/v1/secret/data/payments/api | jq .wrap_info

$ curl -s -X POST -d '{"token":"<wrapping_token>"}'
$VAULT_ADDR/v1/sys/wrapping/lookup | jq .data
$ curl -s -X POST -H "X-Vault-Token: <wrapping_token>"
$VAULT_ADDR/v1/sys/wrapping/unwrap | jq .data
```

From Python:

```
# wrap_demo.py
import os
import hvac

sender = hvac.Client(url=os.environ["VAULT_ADDR"],
token=os.environ["VAULT_TOKEN"])
wrapped = sender.read("secret/data/payments/api", wrap_ttl="5m")
wrap_token = wrapped["wrap_info"]["token"]
print("hand this over:", wrapped["wrap_info"]["accessor"],
wrapped["wrap_info"]["creation_path"])

# Recipient: authenticate with the wrapping token itself, unwrap exactly once
recipient = hvac.Client(url=os.environ["VAULT_ADDR"], token=wrap_token)
secret = recipient.sys.unwrap()
print(secret["data"]["data"])          # {'key': 'pay-123'}
```

Where wrapping is used

Scenario	What gets wrapped	Who unwraps
AppRole secure introduction	`secret_id` from `auth/approle/role/<r>/secret-id`	The application at start-up
VM bootstrap with Terraform/cloud-init	A token or `secret_id`	The VM's first-boot script
Onboarding a new admin	Initial userpass password	The new admin

Scenario	What gets wrapped	Who unwraps
One-off credential for a vendor	Arbitrary data via `sys/wrapping/wrap`	The vendor, once
Enforced by policy	`min_wrapping_ttl` / `max_wrapping_ttl` on a path (Module 2)	Whoever is authorised

Good practice: keep wrap TTLs short (minutes), always check `creation_path` before unwrapping, and alert on any unwrap failure for a token you issued.

Capstone lab — Day 2 end to end

> **Do Lab 2.9 — Day 2 capstone** (45 min). A payments app starts with only a `role_id` and a wrapped `secret_id`; it verifies and unwraps the `secret_id`, logs in with AppRole, reads its config, gets a 15-minute PostgreSQL user, runs a query, and revokes everything on exit. The setup script, the app, expected output and the self-check list are in the Lab Guide.

Troubleshooting, cheat sheet and review

Day 2 failures usually trace back to one of three gaps: Vault's own credential lacks permission on the target system, a role's bindings don't match the caller's identity, or a policy path doesn't match the API path.

Troubleshooting

Error message (abridged)	Likely cause	Fix
`"<role>" is not an allowed role`	Role missing from the connection's `allowed_roles`	`vault write database/config/<conn> allowed_roles=...`
`permission denied to create role`	Vault's DB admin lacks `CREATEROLE`	`ALTER ROLE vaultadmin CREATEROLE;`
`role "v-..." cannot be dropped because some objects depend on it`	Revocation SQL drops the role without revoking grants	Add `REVOKE ALL ...` (or `REASSIGN OWNED`) before `DROP ROLE`
`failed to verify connection ... connection refused`	Vault cannot reach the DB host/port (common when Vault runs in Docker or Kubernetes)	Use a reachable hostname, not `127.0.0.1`
AWS `InvalidClientTokenId` just after issue	IAM eventual consistency	Retry after a few seconds, or prefer `assumed_role`
Azure `AuthorizationFailed`	RBAC assignment still propagating, or Vault's SP lacks rights	Wait a minute; check Graph permission and Owner/UAA on scope
AppRole `invalid secret id`	Expired, uses exhausted, or wrong role	Mint a new `secret_id`; check `secret_id_ttl` and `secret_id_num_uses`

Error message (abridged)	Likely cause	Fix
`source address ... unauthorized through CIDR restrictions`	Caller IP outside `secret_id_bound_cidrs` / `token_bound_cidrs`	Fix the CIDR list or the network path
Kubernetes login: `service account name not authorized`	`bound_service_account_names` or namespaces don't match the pod	`kubectl get pod -o jsonpath='{.spec.serviceAccountName}'` and compare
`tokenreviews.authentication.k8s.io is forbidden`	Reviewer service account lacks `system:auth-delegator`	Create the ClusterRoleBinding for Vault's reviewer SA
JWT: `error validating claims` / `aud claim does not match`	`bound_claims` or `bound_audiences` differ from the token	Decode the JWT and compare claims; align `jwtGithubAudience`
`lease is not renewable` / `lease not found`	Already expired, or a non-renewable type (e.g. STS creds)	Request a new credential
`wrapping token is not valid or does not exist`	Already unwrapped or expired	Treat as a possible interception; revoke and re-issue
`403 permission denied`	Policy path or capability mismatch	`vault token capabilities <token> <path>` and `-output-policy`

```
# Debugging toolkit
$ vault token lookup                                    # policies, TTL,
identity of the current token
$ vault token capabilities <path>                       # what this token may
do on a path
$ vault read -format=json sys/internal/ui/resultant-acl | jq .data   # full
effective ACL
$ vault <command> -output-policy                        # the policy a command
needs
$ vault read database/config/<conn>                     # connection settings
(password hidden)
$ vault read auth/kubernetes/role/<role>                # bindings to compare
with the pod
$ echo '<jwt>' | cut -d. -f2 | base64 -d 2>/dev/null | jq   # inspect JWT
claims
$ vault monitor -log-level=debug                        # live server log via
the API
```

Cheat sheet

Task	Command
Enable DB engine + connection	`vault secrets enable database` · `vault write database/config/<c> plugin_name=... connection_url=... allowed_roles=...`

Task	Command
Dynamic DB role / creds	`vault write database/roles/<r> db_name=<c> creation_statements=@file.sql default_ttl=1h max_ttl=24h` · `vault read database/creds/<r>`
Static DB role	`vault write database/static-roles/<r> username=<u> rotation_period=24h` · `vault read database/static-creds/<r>`
Rotate root	`vault write -f database/rotate-root/<c>` · `vault write -f aws/config/rotate-root`
AWS creds	`vault read aws/creds/<role>` · `vault write aws/sts/<role> ttl=30m`
Azure creds	`vault read azure/creds/<role>`
Policies	`vault policy fmt` · `vault policy write <n> <file>` · `vault token capabilities <t> <path>`
AppRole	`vault read auth/approle/role/<r>/role-id` · `vault write -wrap-ttl=120s -f auth/approle/role/<r>/secret-id` · `vault write auth/approle/login role_id=... secret_id=...`
Kubernetes auth	`vault write auth/kubernetes/config kubernetes_host=...` · `vault write auth/kubernetes/role/<r> bound_service_account_names=... bound_service_account_namespaces=...`
JWT / OIDC	`vault write auth/jwt/config oidc_discovery_url=...` · `vault login -method=oidc role=<r>`
Leases	`vault lease lookup` · `vault lease renew -increment=1h` · `vault lease revoke [-prefix]`
Mount TTLs	`vault secrets tune -default-lease-ttl=30m -max-lease-ttl=2h <mount>/`
Cubbyhole	`vault write cubbyhole/<k> ...` · `vault read cubbyhole/<k>`

Task	Command
Wrapping	`<any command> -wrap-ttl=5m` · `vault write sys/wrapping/lookup token=...` · `vault unwrap <token>` · `vault write -wrap-ttl=10m sys/wrapping/wrap k=v`

Review quiz

1. Name two security advantages of a dynamic database credential over a static one.
2. Why does the PostgreSQL revocation SQL need `REVOKE` before `DROP ROLE`?
3. What does `vault write -f database/rotate-root/appdb` change, and what is the risk in a lab?
4. Which AWS credential type creates no IAM users, and how do you request it?
5. A token has `read` on `secret/data/app/*` and only `list` on `secret/data/app/config`. Can it read `secret/data/app/config`? Why?
6. What does the `+` wildcard match in a policy path?
7. Which policy setting forces a `secret_id` to be fetched only in wrapped form?
8. Which part of an AppRole login is secret, and how should it be delivered?
9. In Kubernetes auth, which API does Vault call to validate a pod's token, and what two bindings does the role check?
10. What stops any GitHub repository from using your `gha-deploy` JWT role?
11. A lease has TTL 1h and max TTL 4h. At 3h30 the app renews with `-increment=1h`. What TTL does it get?
12. A recipient's `vault unwrap` fails with "wrapping token is not valid". What should they assume, and why?

Answers

1. Any two of: unique per requester, short-lived and auto-revoked, precise audit of who got which credential, small blast radius, revocable per lease or prefix.
2. PostgreSQL refuses to drop a role that still holds privileges on objects; revoking first removes those dependencies.
3. It changes Vault's admin password on the database so only Vault knows it; in a lab, the documented password stops working.
4. `assumed_role` — `vault write aws/sts/<role> ttl=30m` (or `vault read aws/creds/<role>`).
5. No. The most specific path rule wins, and the exact `secret/data/app/config` rule grants only `list`.

6. Exactly one path segment, e.g. `secret/data/+/config` matches `secret/data/team1/config` only.
7. `min_wrapping_ttl` (with `max_wrapping_ttl`) on the `auth/approle/role/<r>/secret-id` path.
8. The `secret_id`; deliver it short-lived, single-use and response-wrapped through a trusted orchestrator, separately from the `role_id`.
9. The TokenReview API; `bound_service_account_names` and `bound_service_account_namespaces`.
10. `bound_claims` (repository and ref) together with `bound_audiences`.
11. 30 minutes — renewals cannot extend past the 4h max TTL.
12. That the token may have been intercepted and used by someone else (or expired); it is single-use, so they should alert and have the secret revoked and re-issued.

Coming on Day 3 — PKI, Transit, Enterprise & Operations: the PKI engine (root and intermediate CA, certificate issuance), the Transit engine for encryption as a service, auto-unseal with cloud KMS and seal wrapping, the full Vault Agent feature set, high availability, replication and disaster recovery, monitoring and audit devices, basic troubleshooting, Terraform and Ansible automation, and implementing Vault on Kubernetes — ending with a capstone that joins PKI, Transit and Vault Agent. See the Day 3 Study Guide and the Day 3 Lab Guide.